

DIGDESIGN – Digital Design

Arduino's Sonic Scanner: A Radar Project

Prepared by:

Jericho Canlas

Rayster Landingin

Ramilo Lising

Arvi Joshua Mariano

Calvin Ken Palasigue

Karl Christian Panigbatan

BSCS -3B

Submitted To:

Rengel V. Corpuz

Instructor

May 15, 202

Table of Contents

I.	INTRODUCTION.....	3
	a. Project Content.....	3
	b. Objectives.....	4
II.	SCOPE AND LIMITATION.....	6
	a. Conceptual Framework.....	7
III.	BACKGROUND OF THE STUDY.....	9
	a. Related Studies.....	9
	b. Feature Matrix.....	11
	c. Hardware Specifications.....	12
	d. Software Specifications.....	14
	e. System Architecture.....	16
IV.	METHODOLOGY.....	17
V.	PROJECT TIMELINE.....	20
VI.	CHALLENGE FACED.....	22
VII.	SOLUTION.....	23
VIII.	RESULTS.....	24
IX.	CONCLUSION.....	31
X.	RECOMMENDATIONS.....	32
XI.	REFERENCES.....	34

I. INTRODUCTION

Arduino's Sonic Scanner: A Radar Project introduces an accessible and budget-friendly radar system built using Arduino technology. This project offers enthusiasts an opportunity to delve into the world of radar technology through hands-on construction and experimentation.

Project Content

The overview provides a foundational understanding of radar technology, including its principles and applications across various industries. This section serves as a primer, setting the stage for the hands-on exploration to follow.

Component Identification is a crucial step where participants learn to identify and source the necessary components for the Sonic Scanner project. From Arduino microcontrollers to ultrasonic sensors, each component plays a vital role in the construction of the radar system. Assembly Instructions guide participants through the step-by-step process of building the radar scanner. Instructions ensure that participants can confidently connect the various modules and configure the system for optimal performance. Programming is an essential aspect of the project, where participants learn to program the Arduino microcontroller. Topics covered include data acquisition, signal processing, and user interface development, providing participants with valuable coding skills. Testing and Calibration are crucial steps to ensure the accuracy and reliability of the Sonic Scanner. Participants learn techniques for testing the system in different environments and calibrating it for optimal performance.

Applications and Extensions explore the diverse applications of the radar system, from object detection to distance measurement. Participants are encouraged to experiment

with additional features and functionalities, fostering creativity and innovation. Community Resources provide participants with access to online forums, tutorials, and project repositories for further learning and collaboration. Engaging with the community allows participants to share experiences and learn from fellow enthusiasts. By following the project content outlined above, participants gain a comprehensive understanding of radar technology while developing practical skills in electronics, programming, and hardware assembly. Arduino's Sonic Scanner offers an engaging and educational journey into the fascinating world of radar systems.

Objectives

The objective of this project is to develop an innovative system titled "Arduino Sonic Scanner: A Radar Project." This project aims to explore the practical applications of sonic scanning technology using an Arduino-based system equipped with ultrasonic sensors. Specifically, we aim to create a radar sonic scanner that detects the presence of people within close range. Upon detection, the system will emit an audible alert to warn individuals of potential dangers, such as proximity to an electrical fence. This project will demonstrate the effectiveness and versatility of Arduino and sonic sensor technology in enhancing safety through timely alerts. Specifically, the key objectives of this study are the following:

- **Distance Display** - To provide a display of the measured distance for easy monitoring.
- **Detect Object Presence and Distance** - detect the presence of an object and measure its distance from the sensor.
- **Audible Alert**: When an object is detected within the specified range, the buzzer will activate, providing an immediate sound alert to warn of potential danger.
- **Efficiency** - To improve object tracking and the actual time that it takes a process to occur monitoring by removing delays and increasing the speed at which objects are detected.

- **Cost-Effectiveness** - To determine a useful application for Arduino microcontrollers and ultrasonic sensors for radar initiatives presenting a cost substitute, to proper radar setups.

To achieve these key objectives, the Arduino Sonic Scanner project will involve the development of multiple important elements, includes:

- **Ultrasonic Sensor Integration** - To ensure the accurate measurement distance and detect objects, this module will focus on using ultrasonic sensors in combination with an Arduino microcontroller.
- **Algorithm Development** - The development of algorithms that manage the data from the ultrasonic sensors will be involved by this module, making it accurate for the system to accurately detect and track objects.
- **System Testing and Optimization** - This module will involve testing the system under many conditions and enhancing its performance to ensure good and accurate operation.

II. SCOPE AND LIMITATIONS

In this project, "Arduino's Sonic Scanner: A Radar Project," focused on building a radar system that can detect objects and display the results on a screen. This gives us a chance to learn about radar technology and how to build it using Arduino boards and sensors. But the project has its limits and the areas it covers.

Scope:

This radar system will be capable of detecting objects (1-120cm) within a specified range(80cm) it will emit a sound to alert people of the danger and display their locations on a screen. The team will learn how to connect hardware components, such as Arduino boards and sensors, and write code to integrate these elements effectively. Additionally, they will have the opportunity to explore various applications of radar technology and engage in creative projects.

Limitations:

The system might not be as accurate or powerful as professional radar systems. It could struggle to detect distant objects or distinguish them with high precision. Additionally, assembling all the components might be challenging, especially for those new to this kind of technology. Furthermore, sourcing all the necessary parts could be difficult, depending on location and budget constraints.

CONCEPTUAL FRAMEWORK

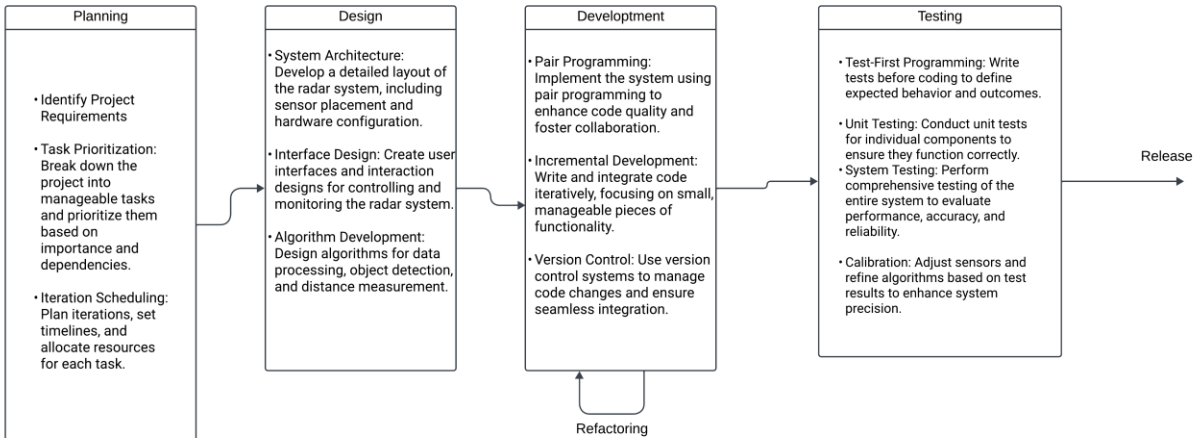


Figure 1: Conceptual Framework

The conceptual framework for "Arduino's Sonic Scanner: A Radar Project" consists of four key phases: Planning, Design, Coding, and Testing.

Planning:

Conduct thorough research to define project scope, objectives, and necessary resources.

Design:

Collaboratively create a detailed blueprint for the radar system, defining architecture, functionality, and interface.

Development:

Write code to implement system functionalities, ensuring adherence to standards and best practices.

Testing:

Rigorously test the system for functionality and reliability through various testing methods.

Each phase is essential for project success, guiding development from inception to completion.

III. BACKGROUND OF THE STUDY

The study focuses on the development of an innovative system titled "Arduino Sonic Scanner: A Radar Project." This project aims to explore the capabilities of sonic scanning technology within the context of A radar project. By utilizing Arduino technology and sonic sensors, the research delves into practical applications and implications of radar systems in a hands-on and educational setting.

Related Studies

RADAR, or Radio Detection and Ranging, finds diverse applications across various domains such as military installations and commercial sectors. Operating on the principle of emitting and receiving electromagnetic waves, RADAR systems excel in detecting and analyzing various physical components including distance, speed, position, range, direction, and size, regardless of whether the targets are stationary or in motion. Over time, RADAR technology has witnessed significant advancements, particularly in the field of navigation, where it plays a pivotal role in enhancing safety, efficiency, and precision in transportation and exploration endeavors. [1]

Radars have historically served as crucial tools for detecting distant objects and putting them on display. In contemporary times, their application spans across various military and aerial contexts, showcasing their widespread utility. One common technique involves the utilization of ultrasonic waves to mimic the functionalities of sonar or radar sweeps. These waves are emitted from a sensor, which is often mounted on a motorized mechanism capable of creating sweeping angles and rotating the sensor. The data collected through this process is then transmitted to a screen display, where it is rendered

in the form of graphical radar blips, offering users a clear and comprehensive visualization of the surrounding environment. [2]

Ultrasonic sensors play a pivotal role in detecting objects along the path of a radar system. These sensors are typically mounted on a servo motor, enabling them to rotate and scan across various directions in a rotational motion. This comprehensive coverage is crucial for ensuring effective object detection. To facilitate this process, specialized processing software is employed, which interprets the data collected by the ultrasonic sensor and translates it into actionable information. The versatility of ultrasonic sensors extends to numerous electronics projects, where they are utilized for tasks such as distance measurement and object detection. In the radar system, the ultrasonic sensor serves as a key component for identifying objects within its range, while the servo motor aids in covering distances by enabling controlled rotation. This integration of technologies ensures the radar system's ability to accurately perceive and respond to its surroundings. [3]

Using radio waves, radar is a detecting device that can measure an object's distance, height, direction, or speed. It can identify a wide range of things, including ships, airplanes, and weather patterns. For radar to function, radio wave or microwave pulses must be sent out. These pulses then reflect off objects and return some of the wave's energy to an antenna or dish close to the transmitter.[5]

FUNCTIONALITY AND FEATURE MATRIX

This feature matrix helps to compare the different radar systems based on their capabilities. It shows which features each system possesses and highlights the unique features of your radar system compared to others.

Functionality / Feature	Arduino Radar System with Ultrasonic Sensor on a Breadboard	Basic Ultrasonic Sensor-Based Radar	Ultrasonic Sensor-Based Radar with Limited Angle Scanning:
Object Detection	✓	✓	✓
Distance info	✓	✓	✗
Angle info	✓	✗	✗
Real-time Object Detection	✓	✓	✓
Sound Alert	✓	✗	✗

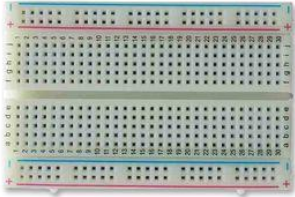
Table 1: Functionality and Feature Matrix

HARDWARE AND SOFTWARE SPECIFICATIONS

Hardware Requirements

These hardware components form the foundation of the radar system, enabling its development, functionality, and testing. They work in concert to detect objects, measure distances, control movement, and process data, allowing for the creation of an effective and versatile radar system.

Name	Image	Description
Computer /Laptop		Used for programming the Arduino microcontroller, running the radar system, and interfacing with the radar scanner during testing and calibration.
Arduino Board		An Arduino Uno R3 microcontroller board, featuring an ATmega328P processor, with digital and analog I/O pins, flash memory, and clock speed of 16 MHz. It serves as the central control unit for the radar system.

Buzzer		Buzzer is a device that produces sound when an electric current is passed through it.
Ultrasonic Sensor		An HC-SR04 ultrasonic sensor, operating at 5V with a detection range of 2cm to 400cm and an accuracy of $\pm 3\text{mm}$. It is responsible for object detection and distance measurement in the radar system.
Servo Moto		A SG90 servo motor, operating at 4.8V to 6V, with a speed of 0.10 sec/60°, and an operating angle of 180°. It controls the movement or scanning capabilities of the radar system.
Breadboard		A solderless prototyping board with a grid of holes for building and testing electronic circuits. It allows for quick and temporary circuit assembly without

		soldering, making it ideal for experimentation and prototyping.
Jumper Wires		Flexible wires with connectors at each end, commonly used to make connections on a breadboard or between electronic components. They come in various lengths and colors, facilitating easy and temporary connections in prototyping and testing electronic circuits.

Table 2: Hardware Requirements

Software Requirements

Software tools provide the necessary environment for developing, testing, and documenting the radar system. They enable programmers to write and debug code, process signals, manage project files, and communicate project information effectively.

Arduino IDE		Essential software tool for programming the Arduino microcontroller to enable data acquisition, signal processing, and user interface development for the radar system.
-------------	---	---

Processing		Processing is a free graphics library and integrated development environment (IDE) built for the electronic arts, new media art, and visual design communities with the purpose of teaching non-programmers the fundamentals of computer programming in a visual context.
Windows Operating System		Compatible with Windows 7, Windows 10, and Windows 11 for running the Arduino IDE and signal processing libraries.
Google Chrome		Web browser used for accessing online resources, forums, or tutorials related to the Arduino Sonic Scanner project.

Table 3: Software Requirements

System Architecture

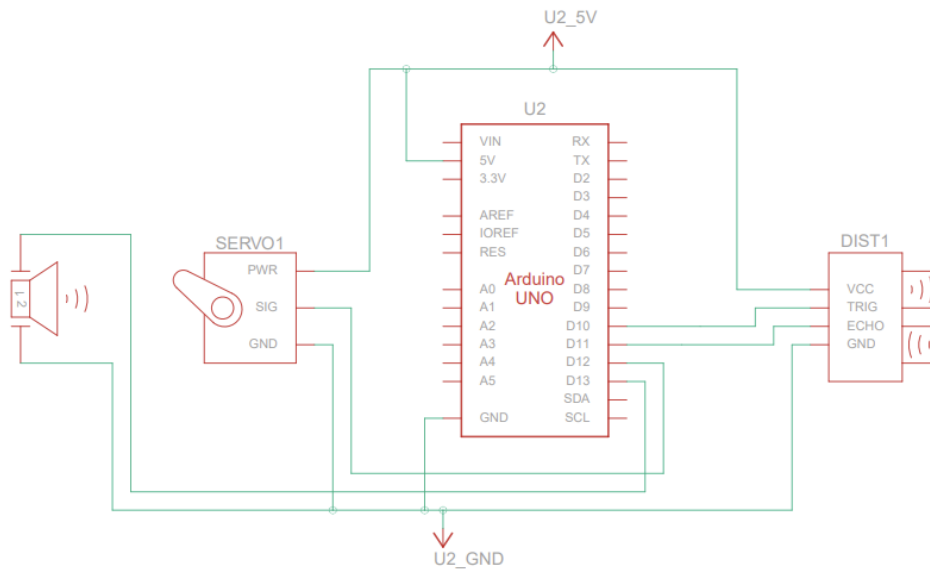


Figure 2: Schematic Diagram

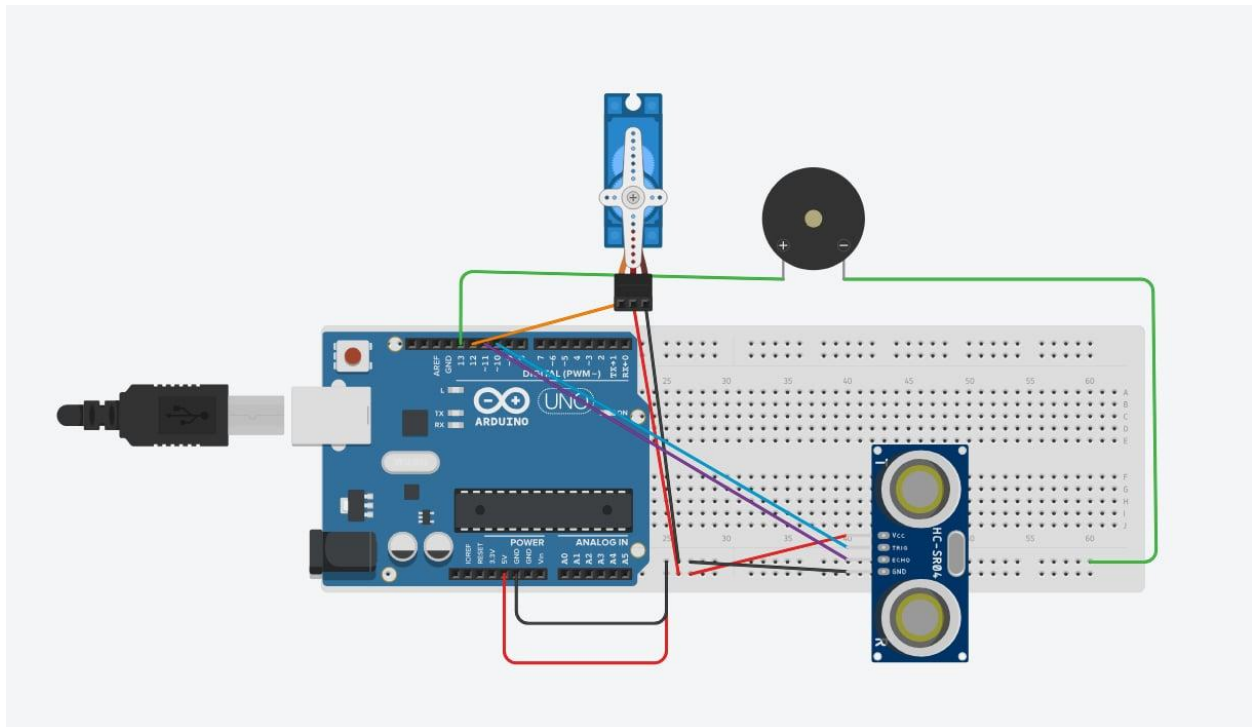


Figure 3: Tinkercad

IV. METHODOLOGY

Research was conducted on developing an Arduino Radar System with an ultrasonic sensor on a breadboard. The chosen methodology relied on web-based resources for information gathering. Online forums, tutorials about Arduino, and projects guided the team to build the entire system. Search questions focusing on keywords like “Arduino Radar System” and “Ultrasonic Sensor with Arduino” helped pinpoint relevant information for the foundation of their research. However, using web-based resources offered immense information, but this approach presented limitations. The accuracy of online information could vary, necessitating prioritization of established forums and credible websites. Web resources might not comprehensively cover the whole project aspect, potentially overlooking design, and troubleshooting steps might be hard to identify. Relying on online resources limits the ability to collect more valuable information from textbooks, and past projects. To mitigate these limitations, the researchers prioritized collecting information from official Arduino websites, credible electronic forums and recognized project repositories. Putting up all the collected information from different sources, served as a foundation for the research. To ensure consistency the team tried to collect information from electronic textbooks for better understanding of the project. Finally, constructing the Arduino Radar System and testing its functionality would provide valuable insights for future projects.

Implementing Extreme Programming (XP) principles in the development of Arduino’s Sonic Scanner project holds the promise of significantly enhancing adaptability, quality, and the overall experience of the development team. By fostering a collaborative

environment through pair programming, developers can engage in continuous code review, knowledge sharing, and faster problem resolution, ultimately leading to higher quality code and improved learning opportunities. Prioritizing test-first programming ensures that the radar system behaves as expected, with automated tests identifying and rectifying defects early in the development process. Breaking down the project into small releases allows for regular delivery of working features, enabling stakeholders to provide timely feedback, and facilitating iterative improvements. Continuous integration streamlines the development process by automating build, test, and deployment tasks, reducing integration issues, and ensuring code reliability. Whole-team collaboration fosters open communication, feedback sharing, and collective decision-making, maximizing the value delivered by the radar system. An informative workspace and weekly iteration cycles provide visibility into project progress, priorities, and key metrics, maintaining momentum and motivation. By promoting a healthy work-life balance and energized work environment, the development team can sustain productivity and prevent burnout, ultimately contributing to the successful development of a robust and user-friendly DIY radar system.[4]

Extreme Programming (XP)

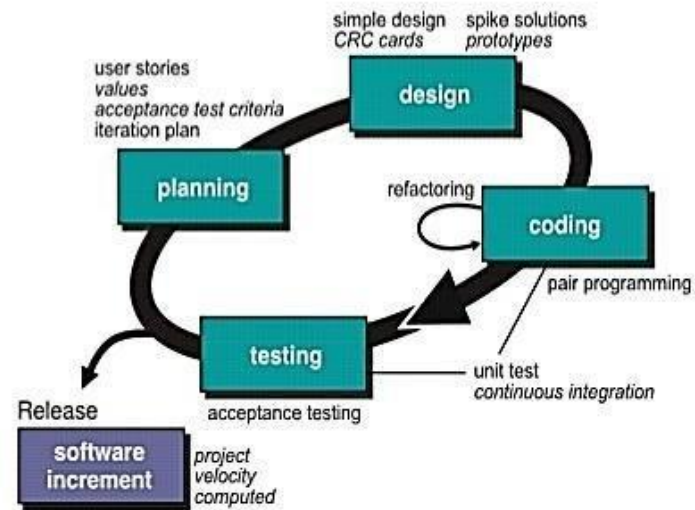


Figure 4: Extreme Programming

V. PROJECT TIMELINE

This section includes the chronological schedule of the execution of the project, outlining the key phases of the system development process, starting with the initial phases followed by the next stages.

Planning:

The project kicks off with a thorough planning phase, where delve into researching existing radar or sonar systems, Arduino capabilities, and sensor options relevant to the project. By gathering information and studying available technologies, the aim is to understand the project scope, identify potential challenges, and determine the best approaches for system development. Through effective planning, set clear objectives and lay the groundwork for a successful implementation.

Design:

Following the planning phase, transitioning into the design stage. This collaborative effort involves creating a detailed blueprint for our project, outlining its requirements, architecture, and components. Designing the layout of the system's user interface, deciding on the placement of sensors, the design of the display interface, and the overall system functionality. By meticulously planning the design, it is ensured that all stakeholders are aligned and that the subsequent development stages proceed smoothly.

Development:

With the design in place, moving to the coding phase, where the project is brought to life. This involves writing code and implementing features to build the Arduino-based radar system according to the design specifications. The team develops code for sensor data acquisition, signal processing, and display output, ensuring that each component functions seamlessly within the system. Through rigorous coding practices and adherence to best practices, the team strive to deliver a high-quality solution that meets our project objectives.

Testing:

As near completion of the development phase, entering the critical testing stage. Here, they conduct comprehensive testing to validate the functionality and performance of our radar system. They start with unit testing, where individual components are tested in isolation to identify any bugs or issues. Subsequently, they progress to integration testing, ensuring that all components work together as intended. Finally, they perform system testing to evaluate the overall performance, efficiency, and accuracy of our radar system. Through meticulous testing and quality assurance efforts, the aim is to deliver a solution that meets the needs of our stakeholders.

VI. CHALLENGES FACED

The development team faced a wide range of obstacles in the process of developing Arduino's Sonic Scanner: A Radar Project from technical and logical challenges.

Technical Challenges:

- **Sensor Accuracy:** Ultrasonic sensors may provide inconsistent distance measurements, particularly in different environmental conditions.
- **Data Processing:** Managing and interpreting real-time sensor data efficiently can be complex, especially for beginners.
- **Motor Control:** Achieving precise and smooth control over servo motors for sweeping motions can be technically demanding.

Logistical Challenges:

- **Component Compatibility:** Ensuring compatibility between various components such as Arduino boards, sensors, and motors is crucial.
- **Resource Availability:** Sourcing required components and tools may be challenging, depending on local availability and budget constraints.
- **Documentation and Support:** Finding comprehensive documentation and community support for DIY radar projects can be essential for success.

VII. SOLUTION

In response to the challenges that have been encountered, the development team implemented a comprehensive set of solutions, demonstrating the commitment to overcome the challenges and providing a reliable facial recognition attendance system.

Technical Solutions:

- **Calibration Algorithms:** Develop algorithms to calibrate sensor readings, accounting for environmental factors and enhancing accuracy.
- **Data Filtering:** Implement filtering techniques to process sensor data, reducing noise and improving reliability.
- **PID Control:** Utilize PID (Proportional-Integral-Derivative) control algorithms for precise servo motor control, ensuring smooth sweeping motions.

Logistical Solutions:

- **Thorough Research:** Conduct extensive research to select compatible components and understand their requirements.
- **Supplier Verification:** Verify component availability and reliability from multiple suppliers to ensure consistent access.
- **Online Resources:** Explore online tutorials, forums, and communities for guidance and support during the development process.

VIII. RESULTS

The system development of "Arduino's Sonic Scanner: A Radar Project" yielded significant outcomes, showcasing its efficacy and impact. The radar system was successfully constructed and operated as per the project's objectives, demonstrating the feasibility of building a DIY radar scanner using Arduino technology. It exhibited the capability to detect objects within its designated range, validating its effectiveness in real-world applications. Engaging in the project led to tangible improvements in participants' skills in electronics, programming, and hardware assembly, empowering them for future DIY endeavors.

System Sample

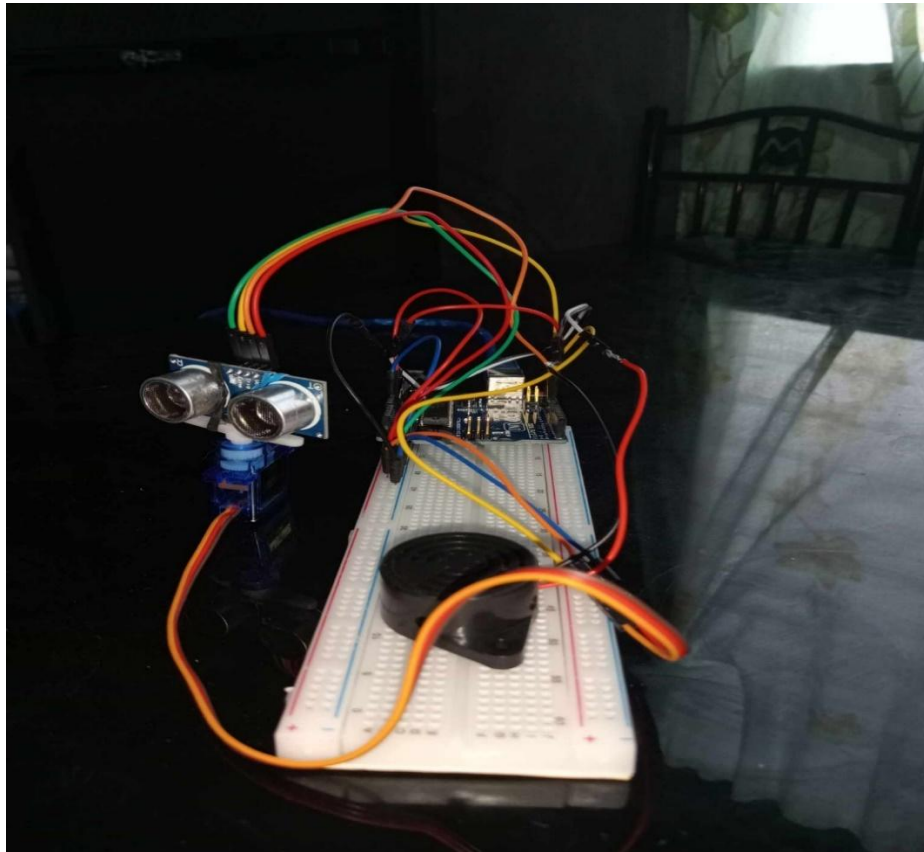


Figure 1. Output 1



Figure 2. Output 3

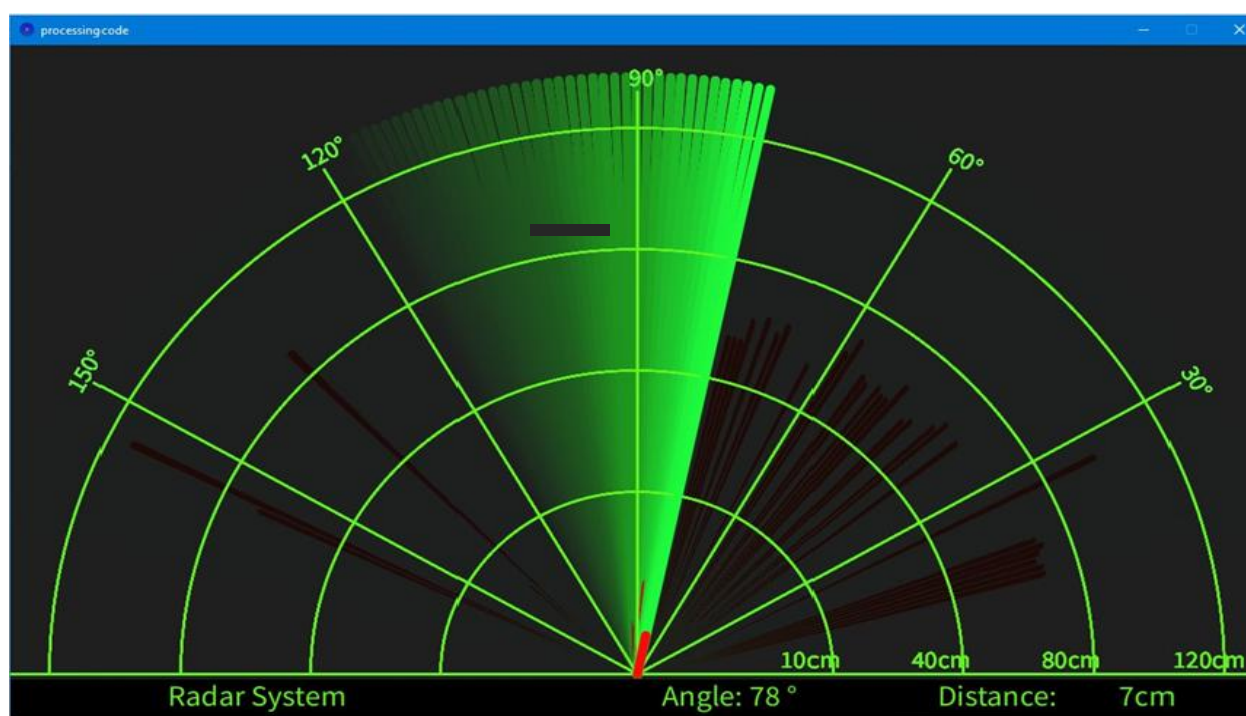


Figure 3. Output 3

Main File Code

```

#include <Servo.h>

const int trigPin = 10;
const int echoPin = 11;
const int buzzerPin = 13; // Positive wire of the buzzer

long duration;
int distance;
Servo myServo;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT); // Set buzzer pin as output

  Serial.begin(9600);
  myServo.attach(12);
}

void loop() {
  for(int i = 15; i <= 165; i++) {
    myServo.write(i);
    delay(30);
    distance = calculateDistance();

    Serial.print(i);
    Serial.print(",");
    Serial.print(distance);
    Serial.print(".");

    if (distance < 50) { // Adjust the distance threshold as needed
      digitalWrite(buzzerPin, HIGH); // Turn on buzzer if an object is detected within 50 cm
    } else {
      digitalWrite(buzzerPin, LOW); // Turn off buzzer otherwise
    }
  }

  for(int i = 165; i > 15; i--) {
    myServo.write(i);
    delay(30);
    distance = calculateDistance();
    Serial.print(i);
    Serial.print(",");
    Serial.print(distance);
    Serial.print(".");

    if (distance < 80) { // Adjust the distance threshold as needed
      digitalWrite(buzzerPin, HIGH); // Turn on buzzer if an object is detected within 50 cm
    } else {
      digitalWrite(buzzerPin, LOW); // Turn off buzzer otherwise
    }
  }
}

```

```

}

int calculateDistance(){
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.034 / 2;
  return distance;
}

```

Processing Code

```

import processing.serial.*;
import java.awt.event.KeyEvent;
import java.io.IOException;

Serial myPort;
String angle = "";
String distance = "";
String data = "";
String noObject;
float pixsDistance;
int iAngle, iDistance;
int index1 = 0;
int index2 = 0;
PFont orcFont;

void setup() {

  size(1200, 700);
  smooth();
  myPort = new Serial(this, "COM3", 9600);
  myPort.bufferUntil('.');
}

void draw() {

  fill(98, 245, 31);
  noStroke();
  fill(0, 4);
  rect(0, 0, width, height - height * 0.065);

  fill(98, 245, 31);
  drawRadar();
  drawLine();
  drawObject();
}

```

```

drawText();
}

void serialEvent(Serial myPort) {

  data = myPort.readStringUntil('.');
  data = data.substring(0, data.length() - 1);

  index1 = data.indexOf(",");
  angle = data.substring(0, index1);
  distance = data.substring(index1 + 1, data.length());

  iAngle = int(angle);
  iDistance = int(distance);
}

void drawRadar() {
  pushMatrix();
  translate(width / 2, height - height * 0.074);
  noFill();
  strokeWeight(2);
  stroke(98, 245, 31);
  arc(0, 0, (width - width * 0.0625), (width - width * 0.0625), PI, TWO_PI);
  arc(0, 0, (width - width * 0.27), (width - width * 0.27), PI, TWO_PI);
  arc(0, 0, (width - width * 0.479), (width - width * 0.479), PI, TWO_PI);
  arc(0, 0, (width - width * 0.687), (width - width * 0.687), PI, TWO_PI);
  line(-width / 2, 0, width / 2, 0);
  line(0, 0, (-width / 2) * cos(radians(30)), (-width / 2) * sin(radians(30)));
  line(0, 0, (-width / 2) * cos(radians(60)), (-width / 2) * sin(radians(60)));
  line(0, 0, (-width / 2) * cos(radians(90)), (-width / 2) * sin(radians(90)));
  line(0, 0, (-width / 2) * cos(radians(120)), (-width / 2) * sin(radians(120)));
  line(0, 0, (-width / 2) * cos(radians(150)), (-width / 2) * sin(radians(150)));
  line((-width / 2) * cos(radians(30)), 0, width / 2, 0);
  popMatrix();
}

void drawObject() {
  pushMatrix();
  translate(width / 2, height - height * 0.074);
  strokeWeight(9);
  stroke(255, 10, 10);

  if (iDistance < 100) {
    pixsDistance = iDistance * ((height - height * 0.1666) * 0.01);
    float xEnd = pixsDistance * cos(radians(iAngle));
    float yEnd = -pixsDistance * sin(radians(iAngle));
    line(0, 0, xEnd, yEnd);
  }
  popMatrix();
}

void drawLine() {
  pushMatrix();
  strokeWeight(9);

```

```

stroke(30, 250, 60);
translate(width / 2, height - height * 0.074);
line(0, 0, (height - height * 0.12) * cos(radians(iAngle)), -(height - height * 0.12) * sin(radians(iAngle)));
popMatrix();'
}

void drawText() {

  pushMatrix();
  if (iDistance > 120) {
    noObject = "Out of Range";
  } else {
    noObject = "In Range";
  }
  fill(0, 0, 0);
  noStroke();
  rect(0, height - height * 0.0648, width, height);
  fill(98, 245, 31);
  textSize(25);
  text("10cm", width - width * 0.3854, height - height * 0.0833);
  text("40cm", width - width * 0.281, height - height * 0.0833);
  text("80cm", width - width * 0.177, height - height * 0.0833);
  text("120cm", width - width * 0.0729, height - height * 0.0833); // Changed from "100cm" to "120cm"
  textSize(30);
  text("Radar System", width - width * 0.875, height - height * 0.0277);
  text("Angle: " + iAngle + " °", width - width * 0.48, height - height * 0.0277);
  text("Distance: ", width - width * 0.26, height - height * 0.0277);
  if (iDistance <= 120) { // Adjusted to show distance only if it's less than or equal to 120cm
    text("          " + iDistance + "cm", width - width * 0.225, height - height * 0.0277);
  }
  textSize(25);
  fill(98, 245, 60);
  translate((width - width * 0.4994) + width / 2 * cos(radians(30)), (height - height * 0.0907) - width / 2 *
sin(radians(30)));
  rotate(-radians(-60));
  text("30°", 0, 0);
  resetMatrix();
  translate((width - width * 0.503) + width / 2 * cos(radians(60)), (height - height * 0.0888) - width / 2 *
sin(radians(60)));
  rotate(-radians(-30));
  text("60°", 0, 0);
  resetMatrix();
  translate((width - width * 0.507) + width / 2 * cos(radians(90)), (height - height * 0.0833) - width / 2 *
sin(radians(90)));
  rotate(radians(0));
  text("90°", 0, 0);
  resetMatrix();
  translate(width - width * 0.513 + width / 2 * cos(radians(120)), (height - height * 0.07129) - width / 2 *
sin(radians(120)));
  rotate(radians(-30));
  text("120°", 0, 0);
  resetMatrix();

```

```
    translate((width - width * 0.5104) + width / 2 * cos(radians(150)), (height - height * 0.0574) - width / 2 *  
sin(radians(150)));  
    rotate(radians(-60));  
    text("150°", 0, 0);  
    popMatrix();  
}
```

IX. CONCLUSION

The development and implementation of "Arduino's Sonic Scanner: A Radar Project" have successfully achieved the set objectives of accuracy, efficiency, and cost-effectiveness in exploring the capabilities of sonic scanning technology through an Arduino-based DIY radar system. This project has not only provided hands-on experience for enthusiasts in radar technology but also contributed to the development of valuable skills in electronics, programming, and hardware assembly.

The systematic approach outlined in the conceptual framework, along with the detailed methodology and project timeline, has ensured a structured and effective development process. Challenges encountered during the project, both technical and logistical, were addressed through innovative solutions, and showcasing adaptability.

In conclusion, the Arduino Sonic Scanner: DIY radar, highlighting its potential applications in object detection and distance measurement within a specified range. The collaboration within the development team, adherence to best practices such as Extreme Programming principles, and utilization of online resources have enriched the project's outcomes and learning experience to make processes with greater efficiency.

X. RECOMMENDATION

Based on the successful implementation of "Arduino's Sonic Scanner: A Radar Project," it is recommended to further explore and expand the project's applications and features. Considerations for future enhancements include:

1. Advanced Signal Processing:

- Explore advanced signal processing techniques to enhance data accuracy and reduce noise in object detection and distance measurement.

2. Integration with Machine Learning:

- Investigate the integration of machine learning algorithms to improve object recognition and classification capabilities, allowing for more sophisticated radar system functionalities.
- Implement neural networks for pattern recognition to differentiate between various object types, enhancing the project's analytical depth.

3. Real-time Data Visualization:

- Implement real-time data visualization techniques to provide a more interactive and informative user interface, enhancing the user experience and usability of the radar system.

4. Community Engagement and Collaboration:

- Foster continued engagement with DIY communities, forums, and project repositories to share experiences, collaborate on enhancements, and contribute to the evolution of DIY radar technology.
- Establish a community-driven platform for users to share their custom modifications, which can inspire further innovation within the project.

XI. REFERENCES

- [1] Ananthula, G. *et al.* (2023) *(PDF) Project Report on Ultrasonic Radar System*. Available at: https://www.researchgate.net/publication/373216250_Project_Report_on_Ultrasonic_Radar_System (Accessed: 11 May 2024).
- [2] *Military spying sonar radar system using Arduino ultrasonic sensor* (2020) Nevon Projects. Available at: <https://nevonprojects.com/arduino-ultrasonic-sonar-radar-monitor-project/> (Accessed: 11 May 2024).
- [3] Shahid, M. (2022) *Arduino radar project using Ultrasonic: Radar using Arduino, Techatronic*. Available at: <https://techatronic.com/radar-using-arduino-ultrasonic-sensor/> (Accessed: 11 May 2024).
- [4] Agile Alliance, “What is Extreme Programming (XP)?,” *Agile Alliance* |, Jun. 14, 2017. [https://www.agilealliance.org/glossary/xp/#:~:text=Extreme%20Programmin,g%20\(XP\)%20is%20an](https://www.agilealliance.org/glossary/xp/#:~:text=Extreme%20Programmin,g%20(XP)%20is%20an)
- [5] T.SAI AVINASH, D.JETTY MARIYAN and CH.KUSHWANT KUMAR (2020) A project report on Arduino radar for OB - I a project report on Arduino Radar for Object Detection, Studocu. Available at: <https://www.studocu.com/row/document/comsats-university-islamabad/eee/a-project-report-on-arduino-radar-for-ob/43601297> (Accessed: 14 May 2024).
- [6] Indian LifeHacker, “How to Make a Radar with Arduino | Arduino Project | Indian LifeHacker,” *YouTube*. Jun. 13, 2017. [YouTube Video]. Available: <https://www.youtube.com/watch?v=JvmlutmQd9U>
- [7] “Arduino Radar Project,” *www.youtube.com*. <https://www.youtube.com/watch?v=kQRYIH2HwfY>
- [8] SciCraft, “Build Your Own DIY Radar System Using Arduino: A Step-by-Step Guide!,” *YouTube*, Apr. 18, 2023. <https://www.youtube.com/watch?v=SvLObGL-5ZY> (accessed May 10, 2024).
- [9] Vector Link, “How To Make A Radar Using Arduino Nano - Vector Link #arduino @vectorlink,” *YouTube*, Jul. 31, 2021. https://www.youtube.com/watch?v=Ab-zD6v_SnY (accessed May 10, 2024).
- [10] Harsh Sharma, “How to Make Radar Using Arduino Uno & Ultrasonic Sensor,” *YouTube*, Jan. 18, 2024. <https://www.youtube.com/watch?v=F3C1LtXdQ7Q> (accessed May 18, 2024).